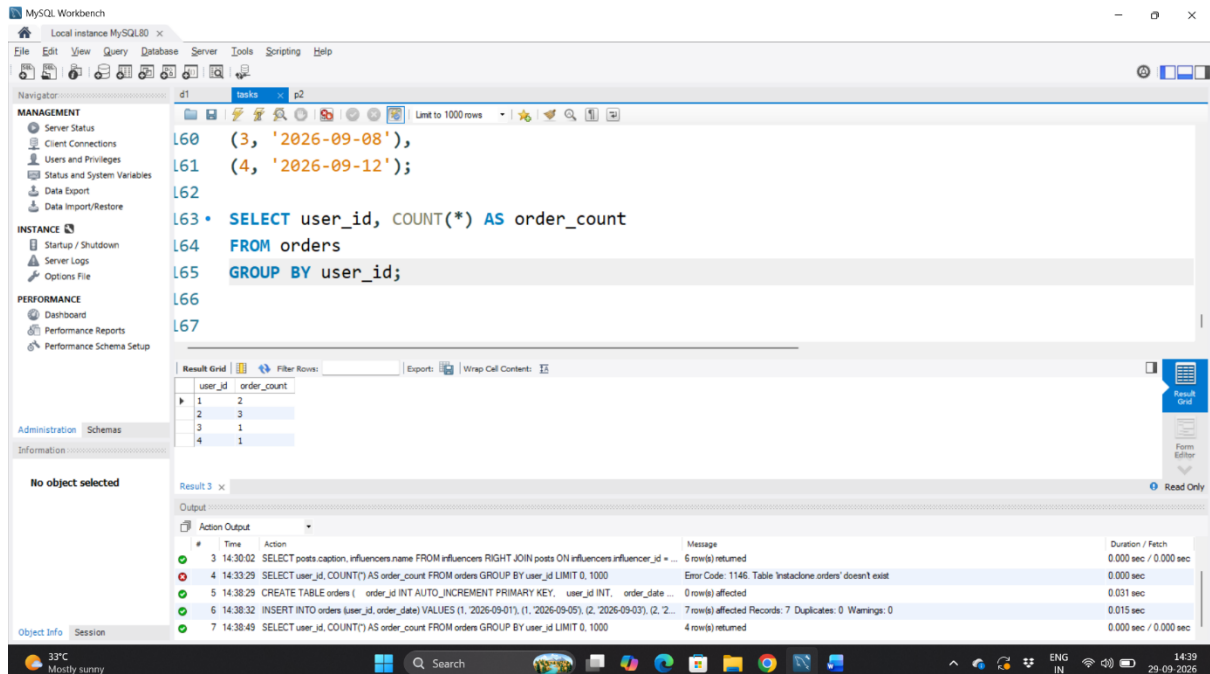


Session 6

Q1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
L60 (3, '2026-09-08'),
L61 (4, '2026-09-12');
L62
L63 • SELECT user_id, COUNT(*) AS order_count
L64 FROM orders
L65 GROUP BY user_id;
L66
L67
```

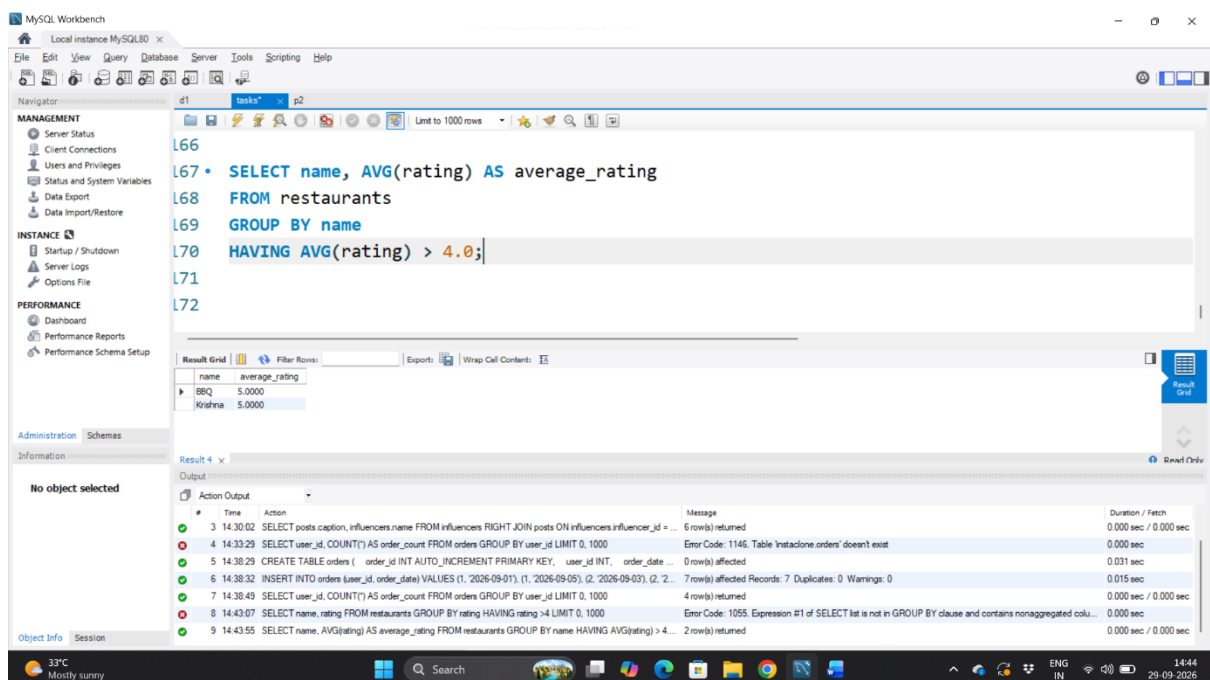
The Results tab shows the output of the query:

user_id	order_count
1	2
2	3
3	1
4	1

The Action Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
3	14:30:02	SELECT posts.caption, influencers.name FROM influencers RIGHT JOIN posts ON influencers.influencer_id = ...	6 row(s) returned	0.000 sec / 0.000 sec
4	14:33:29	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIMIT 0, 1000	Error Code: 1146: Table 'instadone.orders' doesn't exist	0.000 sec
5	14:38:29	CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT, order_date ...	0 row(s) affected	0.031 sec
6	14:38:32	INSERT INTO orders (user_id, order_date) VALUES (1, '2026-09-01'), (1, '2026-09-05'), (2, '2026-09-03'), (2, '2 ...	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.015 sec
7	14:38:49	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Q2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
L66
L67 • SELECT name, AVG(rating) AS average_rating
L68 FROM restaurants
L69 GROUP BY name
L70 HAVING AVG(rating) > 4.0;
L71
L72
```

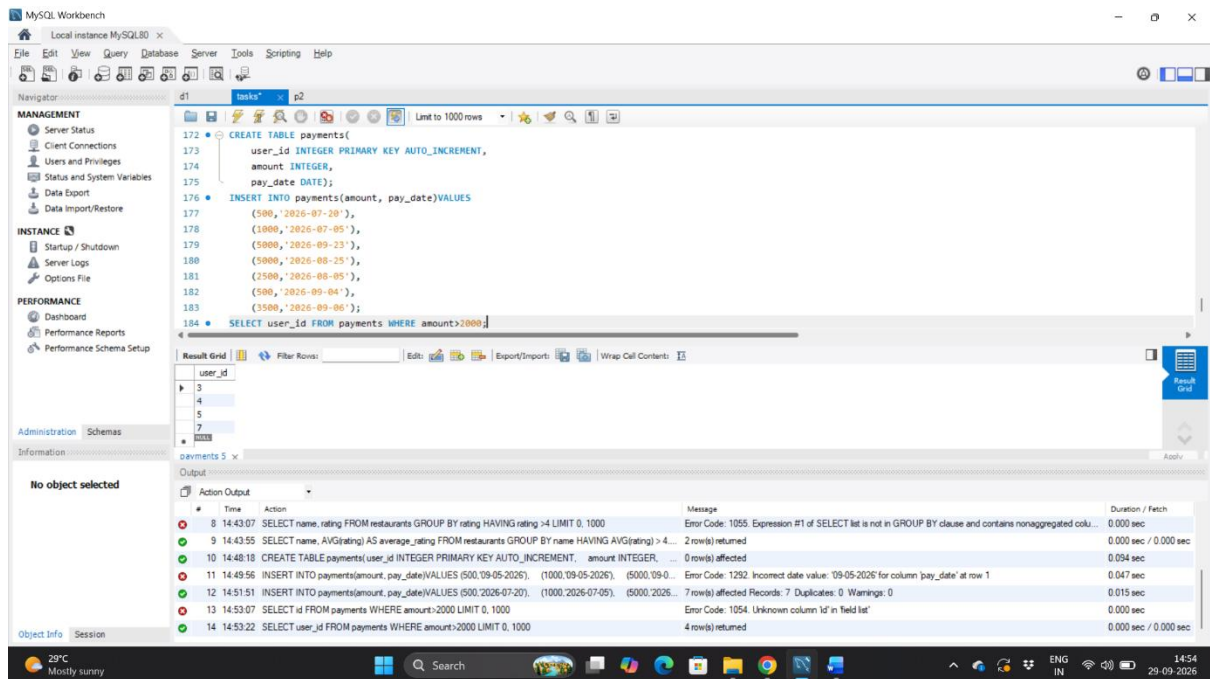
The Results tab shows the output of the query:

name	average_rating
BBQ	5.0000
Krishna	5.0000

The Action Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
3	14:30:02	SELECT posts.caption, influencers.name FROM influencers RIGHT JOIN posts ON influencers.influencer_id = ...	6 row(s) returned	0.000 sec / 0.000 sec
4	14:33:29	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIMIT 0, 1000	Error Code: 1146: Table 'instadone.orders' doesn't exist	0.000 sec
5	14:38:29	CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT, order_date ...	0 row(s) affected	0.031 sec
6	14:38:32	INSERT INTO orders (user_id, order_date) VALUES (1, '2026-09-01'), (1, '2026-09-05'), (2, '2026-09-03'), (2, '2 ...	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.015 sec
7	14:38:49	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
8	14:43:07	SELECT name, AVG(rating) AS average_rating FROM restaurants GROUP BY name HAVING AVG(rating) > 4.0	Error Code: 1055: Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated colu...	0.000 sec
9	14:43:55	SELECT name, AVG(rating) AS average_rating FROM restaurants GROUP BY name HAVING AVG(rating) > 4.0	2 row(s) returned	0.000 sec / 0.000 sec

Q3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.



The screenshot shows MySQL Workbench with a local instance of MySQL8.0. The 'tasks' pane contains the following SQL code:

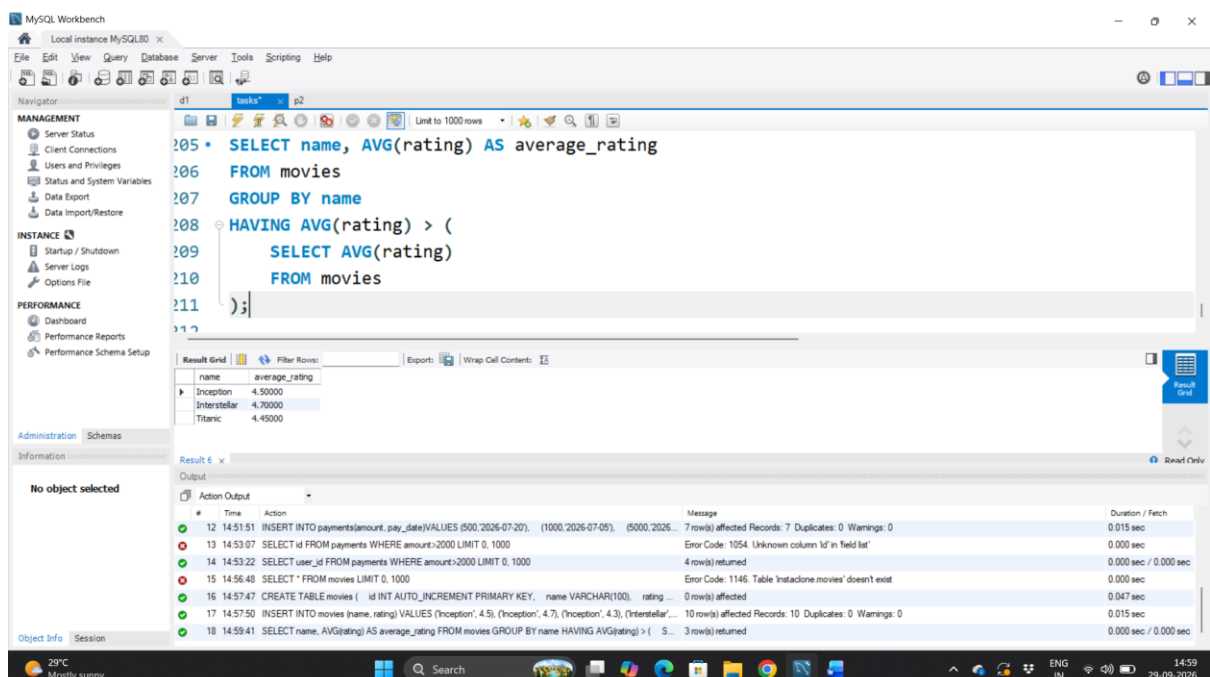
```

172 CREATE TABLE payments(
173   user_id INTEGER PRIMARY KEY AUTO_INCREMENT,
174   amount INTEGER,
175   pay_date DATE);
176 INSERT INTO payments(amount, pay_date)VALUES
177   (500,'2026-07-20'),
178   (1000,'2026-07-05'),
179   (5000,'2026-09-23'),
180   (5000,'2026-08-25'),
181   (2500,'2026-08-05'),
182   (500,'2026-09-04'),
183   (3500,'2026-09-06');
184 SELECT user_id FROM payments WHERE amount>2000;

```

The 'Result Grid' shows the output of the last query, displaying user_ids 3, 4, 5, and 7. The 'Output' pane shows the execution log with various messages and durations.

Q4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies. Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.



The screenshot shows MySQL Workbench with a local instance of MySQL8.0. The 'tasks' pane contains the following SQL code:

```

205 SELECT name, AVG(rating) AS average_rating
206 FROM movies
207 GROUP BY name
208 HAVING AVG(rating) > (
209   SELECT AVG(rating)
210   FROM movies
211 );

```

The 'Result Grid' shows the output of the last query, displaying movie names and their average ratings: Inception (4.50000), Interstellar (4.70000), and Titanic (4.45000). The 'Output' pane shows the execution log with various messages and durations.

